

COLOQUIO ESTRUCTURA DE DATOS

1. ¿Qué es Estructura de Datos, Tipo de Dato y Tipo de Dato Abstracto?

- Estructura de datos: Se puede considerar como un conjunto de variables, quizás de distintos tipos, que se relacionan entre sí y que se pueden operar como un todo. **El componente básico de la estructura de datos es la "celda", las estructuras de datos se implementan a través de los lenguajes y son un modelo que caracteriza y permite almacenar y utilizar una determinada organización de datos.** Son fundamentales para el manejo de información y el desarrollo de sistemas.
- Tipos de datos: Son fundamentales para la implantación de estructuras más complejas. Cuando se define una variable como perteneciente a un tipo de datos se efectúa una abstracción que incluye la especificación de las celdas en que se puede almacenar, el conjunto de valores válidos que ésta celda puede tomar y el conjunto de operaciones que se pueden efectuar. En esencia, un tipo de datos es una colección de valores.
- Tipo de dato abstracto: Un TDA es el resultado de aplicar un proceso de abstracción de datos, reuniendo un Tipo de Dato y las operaciones que se pueden realizar con este tipo de dato. Los TDA son un modelo, y las estructuras de datos son la implementación de estos TDA en un lenguaje de programación.

2. ¿Qué es la abstracción por parametrización?

Es la característica o capacidad que tienen los subprogramas o funciones para mostrarnos sus argumentos como si fuesen variables locales y tratarlas independientemente del valor con la que se las invoque. En este tipo de abstracción la identidad del dato es irrelevante; pero la presencia, el tipo y el número son indispensables.

3. ¿Qué es abstracción por especificación?

El objetivo de este tipo de abstracción es centrar la atención sobre "qué hace", y pasar a un segundo plano el "cómo lo hace". Se utilizan precondiciones, postcondiciones y encabezados de módulos, procedimientos o funciones.

4. ¿Qué son las relaciones entre los elementos de una estructura de datos?

Las relaciones entre los elementos de una estructura establecen la conexión física o lógica entre los distintos elementos que componen la estructura. Los principales tipos de relación son:

- De precedencia: permite representar la secuencia u orden de los elementos.
- De equivalencia: con esta relación es posible equiparar o igualar elementos.
- De jerarquía: permite indicar "niveles de importancia" entre los elementos.
- De pertenencia: determina que algunos elementos están incluidos en otros elementos, conformando de esta manera estructuras más complejas.
- De adyacencia: permite representar la "igualdad de importancia" entre los elementos de la estructura.

Es importante destacar que una estructura puede estar constituida por un conjunto de estructuras elementales o básicas, consecuentemente estas mismas relaciones son válidas entre estructuras.

5. ¿Qué son las funciones básicas y operaciones para estructuras de datos?

Funciones básicas: Son una serie de algoritmos que ejecutan tareas fundamentales que son necesarias para disponer de una estructura de datos.

- Funciones constructora.
- Funciones para acceso.
- Funciones destructoras.

Operaciones: Son las diferentes operaciones que se pueden efectuar sobre una estructura de datos, entre las más importantes están:

- Navegar por la estructura.
- Búsqueda.
- Consulta de la información.
- Copia parcial o total.
- Prueba.
- Modificación.
- Inserción.
- Eliminación.

6. ¿Cómo se relaciona la abstracción de datos y la programación orientada a objetos?

Antes de escribir un programa el desarrollador debe tener una idea bastante clara de cómo lograr la tarea que está programando. Debemos preocuparnos más por lo que el programa debe hacer, no por cómo puede hacerlo. En los lenguajes orientados a objetos un tipo de dato abstracto se llama interface, son parecidas a las clases, pero solamente pueden contener constantes y prototipos de métodos; es decir las especificaciones de los nombres de los métodos, los tipos de parámetros y los tipos de valores de regreso. En otras palabras, una interface es como un "contrato", las clases que implementan una interface "se comprometen" a realizar todo lo que la interface especifica. De esta manera los desarrolladores pueden postergar los detalles específicos de implementación para una etapa posterior del proceso de desarrollo.

7. ¿Cómo se asigna la memoria en un entorno orientado a objetos?

Los lenguajes de programación orientados a objetos ocultan la utilización de punteros, clasifican a los distintos tipos de datos como Data Type, aquellos tipos de datos cuyas variables se almacenan en el stack (clásico modelo de asignación de memoria), y los Reference Type, aquellos tipos de datos que utilizan una variable en el stack (lo que en el modelo no orientado a objetos es el puntero); y una porción de memoria denominado "heap" o "montón" (espacio necesario para los objetos de la aplicación); por supuesto que todos estos lenguajes implementan el recolector de basura, caso contrario se terminaría agotando la memoria del equipo impidiendo la ejecución del programa.

8. ¿Cómo se asigna la memoria en un entorno NO orientado a objetos?

La arquitectura de estos lenguajes de programación utiliza un modelo de asignación de memoria que además los dos segmentos; uno para el código y otro para los datos, incorpora un tercer segmento de memoria llamado "pila" donde se guardan las direcciones de memorias a las que se debe regresar cuando se finaliza un subprograma y también es en ese segmento donde se crean temporalmente las variables locales.

9. ¿Qué es un "constructor" en programación orientada a objetos?

Un constructor es un método que se ejecuta automáticamente cuando se crea un objeto de una clase. El método constructor tiene el mismo nombre que la clase, no se puede indicar un valor de retorno, tampoco devuelve algo. Pero si puede tomar cualquier número de argumentos.

10. Describa el comportamiento de la estructura de datos ArrayList (C# o Java) ¿Cuál es su clasificación?

ArrayList es una estructura de datos que implementa el comportamiento normal de un arreglo. Es decir, el de una colección homogénea de elementos a la que se puede acceder mediante uno o más subíndices.

En C# puede implementar el comportamiento de programación en un solo hilo o programación multihilada o paralela

En java no está preparado para sincronizar su funcionamiento en un ambiente de programación paralela como lo es el vector

11. Describa el comportamiento de la estructura de datos Vector (C++) ¿Cuál es su clasificación?

Básicamente hace lo mismo que el ArrayList de C# o Java, nos permite construir objetos. Esta clase permite crear objetos con bastantes comportamientos. Se clasifican en arreglos estáticos y arreglos dinámicos.

12. Describa el comportamiento de la estructura de datos list (Python) ¿Cuál es su clasificación?

Las listas son semejantes o equivalentes a los arreglos en otros lenguajes. No son un tipo de dato predefinido sino que son un modulo adicional conocido como arreglo eficiente.

13. ¿Qué es el concepto de "lista" en estructura de datos?

Una lista es una secuencia de nodos o de elementos, cuya característica o propiedad estructural principal es que existe un primer elemento y un último elemento. Las agregaciones y extracciones se pueden realizar por cualquier punto de la lista

Se clasifican en:

- Según su forma: lineal, no lineal, abierta y cerrada.
- Según su forma de almacenamiento: almacenada en secuencia, encadenadas.
- Según su forma de acceso: libre, restringida.

Sus operaciones son: agregar y extraer elementos, recorrer obtener información, buscar, copiar, unir, partir.

14. ¿Qué puede decir de las estructuras de datos Pila (stack) y Cola (Queue), respecto de su clasificación?

Pila (stack): Es una lista lineal que se almacena en una serie sucesiva de celdas en. Tiene un comportamiento especial denominado "LIFO", que significa que el último elemento en entrar es el primero en salir, por lo que es una lista restringida. Los elementos se agregan y extraen por el mismo lugar, por lo que es una lista abierta y se puede implementar con una lista almacenada en secuencia o enlazada.

Cola (Queue): Es una lista lineal que se almacena en una serie sucesiva de celdas. Tiene un comportamiento especial denominado "FIFO", que significa que el primer elemento que entra es el primero en salir, por lo que es una lista restringida. Los elementos se agregan por un extremo y extraen por el otro extremo, por lo que es una lista abierta y se puede implementar con una lista almacenada en secuencia o enlazada. En el caso de usar una cola circular se trata de una lista cerrada.

15. ¿Cuál es el problema de la implementación de una cola (Queue) conocida como "cola circular"?

Al utilizar una cola circular, los índices utilizados para indicar por donde extraer el próximo elemento (head) y por dónde agregar uno nuevo (tail), "se persiguen" dando vueltas por el contenedor de la estructura interna, de modo que hace falta un mecanismo para saber cuándo la cola está llena y cuando está vacía.

Al principio, cuando el arreglo está vacío se puede fijar "head igual tail", de manera que al agregar un elemento se lo ubica en la posición indicada por "tail" y luego se "avanza" ese índice. Para retirar el elemento se toma de la posición indicada por "head" y luego se "avanza" ese índice.

En algún momento va a ocurrir que solo queda un lugar disponible de manera que "head" indicará al que está listo para extraerse y "tail" indicará el único lugar disponible en el arreglo. Cuando se agregue un elemento se lo pone en el lugar indicado por "tail" y luego se "avanza" este índice. Cuando esto suceda tendremos que "head es igual tail" y la cola está llena, pero también cuando "head es igual a tail" la cola está vacía.

Tenemos dos alternativas para saber en qué situación estamos:

- 1) Utilizar un variable que cuente los elementos que se agregan y descuenta cuando se extraen.
- 2) Desperdiciar ese último lugar disponible y decir que la cola está llena cuando "el siguiente de tail es igual a head".

La primera de las implementaciones se conoce como cola que prioriza la memoria, esto significa que utiliza toda la memoria que le fue asignada y la segunda de las implementaciones se conoce como cola que prioriza la velocidad porque no necesita incrementar y decrementar una variable que cuenta los elementos dentro de la cola.

16. ¿Qué ventajas y desventajas presenta el manejo de memoria dinámico?

Ventajas:

- El usuario puede disponer de más memoria o devolver lo que no está utilizando.
- Se le puede suministrar memoria a medida que se va necesitando.
- Se pueden implementar varios constructores además del constructor por defecto para poder crear nodos de distinta manera.
- Se facilita el acceso a la estructura interna mediante el uso de getters y setters para cada miembro de la misma.

Desventajas:

- La memoria dinámica es más difícil de manejar ya que se reserva de forma explícita y continúa existiendo hasta que sea liberada, generalmente por parte del programador.
- La memoria dinámica puede afectar el rendimiento.
- Se tienen que llevar a cabo varias tareas, como buscar un bloque de memoria libre y almacenar la posición y tamaño de la memoria asignada, de manera que pueda ser liberada más adelante.

17. ¿Qué problemas puede tener la implementación de la estructura de datos conocida como “arreglos dinámicos” o “arreglos mejorados”?

La operación extraer el primero o agregar al principio es de orden lineal ($O(n)$). También es un problema extraer o agregar en cualquier otra posición (menos el último) porque la operación es de orden lineal.

18. ¿Qué es un "Nodo" en estructura de datos?

Un nodo es una estructura, un registro o un objeto que tiene básicamente dos partes, una de ellas información o datos y la otra al menos un enlace, puntero o referencia a otro. Son herramientas esenciales para la construcción de estructuras de datos dinámicas.

19. ¿Qué problemas presenta una lista encadena simple?

Uno de los problemas es el tiempo de acceso a cualquier elemento ya que es de orden lineal ($O(n)$), mientras más elementos haya en la lista más tiempo se requerirá.

La operación de agregar al final es de orden lineal ($O(n)$) solo si la lista no tiene referencia al último elemento.

La operación de extraer el último elemento es de orden lineal, si o si hay que recorrer la lista hasta el penúltimo elemento para poder desconectar el último.

La operación de extraer o agregar es constante pero el acceso a esa posición es de orden lineal.

20. ¿Que necesita una lista encadena simple o doble para poder almacenar información ordenada?

Una lista encadenada simple o doble necesita que los elementos se comparen entre sí, para ello la lista delega esta responsabilidad a un método. De esta forma cuando queramos almacenar un elemento en orden comparamos este elemento con el primero y según el criterio de ordenación que decidamos, almacenamos el elemento donde corresponda.

21. ¿Cómo se implementan las listas con acceso restringido más conocidas utilizando listas encadenadas?

Pila implementada con lista: basta con utilizar una lista en la estructura interna de la misma y utilizar los métodos que permiten agregar y extraer por la cabeza de la lista; de ese modo se implementa el comportamiento LIFO que las pilas tienen.

Cola implementada con lista: estas también mantienen en su estructura interna un objeto del tipo lista y el comportamiento FIFO se logra agregando por final y extrayendo por la cabeza de la lista.

En ambos casos, tanto la pila (Stack) como cola (Queue) no saben qué tipo de lista están utilizando en su estructura interna, lo que es correcto desde el punto de vista del encapsulamiento; sin embargo por una cuestión de utilizar mejor los recursos (procesador y memoria) es conveniente que se implemente con listas encadenadas simples y en el caso de la cola es necesario contar con enlaces a la cabeza y cola de la lista.

22. ¿Qué es un árbol binario de búsqueda?

Un Árbol Binario de Búsqueda es un árbol binario que tiene los nodos ordenados de alguna manera, en donde, dado un nodo, todos los datos del subárbol izquierdo son menores que los datos de ese nodo, mientras que todos los datos del subárbol derecho son mayores (o iguales) que sus propios datos.

23. ¿Para qué sirve la técnica conocida como "mayor de los menores"?

Esta técnica se utiliza en los árboles binarios para rearmar un árbol cuando se extrae algún elemento que tiene dos descendientes.

Consiste en hallar el nodo cuyo valor es el más grande (mayor) de todos los nodos cuyos valores son menores que el que se desea extraer. Una vez que se encuentra el nodo cuyo valor es el mayor de los menores se procede a "ajustar" los enlaces del árbol y finalmente desconectar el nodo. Así como existe la técnica "mayor de los menores" existe otra técnica muy similar que se conoce como "menor de los mayores" que es lo mismo solo que con la otra rama o descendientes del nodo que se desea extraer.

24. ¿Cuándo se debe realizar un "rebalanceo" o "rotación" en los árboles equilibrados?

Se debe realizar un rebalanceo o rotación en árboles equilibrados cuando la altura del subárbol izquierdo difiere como máximo en uno del subárbol derecho.

25. ¿Por qué se utilizan implementaciones del tipo árbol rojo-negro en pequeñas cantidades de datos (miles de datos)?

Se utilizan árboles rojo-negro en pequeñas cantidades de datos porque la inserción de nuevos datos es más rápida, en promedio hace menos rotaciones, lo mismo ocurre en la eliminación. En cambio en grandes cantidades de datos se pierde tiempo buscando el dato ya que tiene mayor profundidad.

26. ¿Por qué se utilizan implementaciones del tipo árbol AVL en grandes cantidades de datos (millones de datos)?

Se utilizan implementaciones del tipo AVL en grandes cantidades de datos porque la búsqueda de datos es más rápida porque es menos profundo, lo mismo ocurre en la inserción y en la eliminación, excepto en el peor de los casos que es más rápido el árbol rojo-negro.

27. ¿Por qué se utilizan estructuras de datos del tipo árbol para implementar el tipo de dato abstracto conjunto de la matemática?

Se utilizan estructuras de datos del tipo árbol porque para buscar eficientemente un elemento en el conjunto la colección debe estar ordenada, más allá que el conjunto no lo requiera. Luego de realizar esta prueba de existencia se pueden realizar las otras operaciones como unión, intersección, diferencia, y subconjunto.